

Capítulo 12. Métodos supervisados avanzados y ensambles

Los modelos lineales expresan una relación mediante una forma funcional relativamente rígida. Este capítulo estudia métodos capaces de representar interacciones, umbrales y fronteras no lineales sin que el analista deba especificarlas una por una. Esa flexibilidad no elimina la necesidad de formular el problema, construir un protocolo de validación y relacionar las métricas con decisiones reales. Por el contrario, cuanto mayor es la capacidad del modelo, más importante resulta distinguir ajuste de generalización.

Se considerará un conjunto de aprendizaje $D = \{(x_i, y_i)\}_{i=1}^n$, con $x_i \in \mathbb{R}^p$. En clasificación, y_i pertenece a una de K clases; en regresión, $y_i \in \mathbb{R}$. El objetivo empírico general es encontrar una función f que minimice una pérdida regularizada,

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \left[\frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda \Omega(f) \right],$$

donde $\mathcal{F}$ representa la familia de modelos, L cuantifica el desacuerdo con los datos, Ω controla complejidad y λ regula el compromiso entre ajuste y simplicidad. Árboles, máquinas de soporte vectorial y ensambles implementan este principio de maneras diferentes. Las técnicas de explicabilidad permiten estudiar el resultado, pero no convierten por sí solas una asociación predictiva en una explicación causal. Finalmente, el aprendizaje federado modifica el lugar donde se ajusta el modelo, no los fundamentos estadísticos de su evaluación.

Al terminar el capítulo se espera que el lector pueda: derivar criterios de partición y margen; reconocer el papel de la regularización; comparar bagging, Random Forest, boosting y stacking; diagnosticar sobreajuste, inestabilidad y fuga de información; interpretar herramientas globales y locales con sus supuestos; y diseñar conceptualmente un sistema federado consciente de sus riesgos.

12.1. Árboles de decisión

Un árbol de decisión representa una función mediante una secuencia jerárquica de preguntas. Cada nodo interno evalúa una condición, cada rama corresponde a una respuesta y cada hoja produce una predicción. Su atractivo reside en que puede aproximar relaciones no lineales, descubrir interacciones y admitir variables en escalas distintas. Su principal debilidad es la inestabilidad: cambios pequeños en los datos pueden alterar las primeras divisiones y, con ellas, buena parte de la estructura.

12.1.1. Particiones recursivas

Un árbol induce una partición del espacio de atributos en regiones disjuntas $R_1, \dots, R_M$. Para variables numéricas, una división típica de una región R usa el atributo j y el umbral s :

$$R_{\text{izq}}(j, s) = \{x \in R : x_j \leq s\}, \quad R_{\text{der}}(j, s) = \{x \in R : x_j > s\}.$$

El algoritmo voraz examina divisiones candidatas y elige la que más reduce una función de impureza o pérdida. Después repite el procedimiento en cada región hija. Se denomina *recursivo* porque cada hija se trata como un problema del mismo tipo, y *binario* cuando toda división genera dos ramas. El árbol no busca simultáneamente la mejor estructura global: esa optimización combinatoria sería muy costosa. Una elección local óptima puede impedir una estructura posterior mejor.

En clasificación, si p_{mk} es la proporción de la clase k en el nodo m , la predicción de la hoja suele ser $\hat{k}_m = \arg \max_k p_{mk}$. También pueden conservarse las proporciones como estimaciones de probabilidad, aunque las hojas pequeñas producen estimaciones extremas e inestables. En regresión cuadrática, la constante que minimiza la suma de cuadrados en R_m es la media:

$$\hat{c}_m = \arg \min_c \sum_{i: x_i \in R_m} (y_i - c)^2 = \frac{1}{n_m} \sum_{i: x_i \in R_m} y_i.$$

La función resultante es constante por regiones: $\hat{f}(x) = \sum_{m=1}^M \hat{c}_m \mathbf{1}(x \in R_m)$. Esto explica las predicciones escalonadas de un árbol de regresión y su incapacidad para extrapolar tendencias más allá de los valores observados.

Pseudocódigo: crecimiento recursivo

```

CRECER_NODO(datos D, restricciones):
    si se cumple un criterio de detención:
        devolver HOJA con la predicción calculada en D
    para cada atributo j:
        para cada división candidata s permitida:
            separar D en D_izq y D_der
            calcular reducción de impureza
        elegir (j*, s*) con mayor reducción válida
    si ninguna reducción es suficiente:
        devolver HOJA
    devolver NODO(j*, s*,
                  CRECER_NODO(D_izq, restricciones),
                  CRECER_NODO(D_der, restricciones))

```

Los valores ausentes, las variables categóricas de alta cardinalidad y las restricciones de tamaño requieren decisiones explícitas. Imputar antes del ajuste, crear una categoría informativa o emplear divisiones sustitutas son alternativas distintas y deben formar parte del protocolo de validación. Una categoría con muy pocos casos puede generar una regla aparentemente perfecta que no se reproduce.

12.1.2. Entropía y ganancia de información

La entropía de un nodo con distribución de clases $p_1, \dots, p_K$ es

$$H(R) = - \sum_{k=1}^K p_k \log_2 p_k,$$

con la convención $0 \log 0 = 0$. Es cero si todos los ejemplos pertenecen a una clase y alcanza su máximo $\log_2 K$ cuando las clases son equiprobables. En el caso binario,

$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$; es simétrica respecto de $p = 0.5$ y aumenta con la incertidumbre.

Una división de un nodo padre R en hijas $R_1, \dots, R_q$ se evalúa mediante la entropía posterior ponderada:

$$H_{\text{post}} = \sum_{r=1}^q \frac{n_r}{n} H(R_r).$$

La ganancia de información es

$$IG = H(R) - H_{\text{post}}.$$

Esta expresión puede interpretarse como la reducción esperada de incertidumbre sobre la clase al conocer la rama. El peso n_r/n evita que una hija pura pero diminuta domine la decisión.

Supóngase un lote de 100 piezas, 40 defectuosas y 60 conformes. La entropía inicial es $H(0.4) \approx 0.971$ bits. Una división por vibración produce una rama de 30 piezas con 24 defectuosas y otra de 70 con 16 defectuosas. Sus entropías son aproximadamente 0.722 y 0.776. Por tanto,

$$IG \approx 0.971 - [0.30(0.722) + 0.70(0.776)] = 0.211.$$

Otra división debe compararse con la misma base. No basta observar que una rama quedó pura. Además, una variable categórica con muchas categorías puede ofrecer numerosas oportunidades de separar accidentalmente los datos; limitar divisiones, agrupar categorías raras o validar fuera de muestra reduce este sesgo de selección.

La base del logaritmo cambia la unidad, no el orden entre divisiones. La ganancia tampoco es una prueba de asociación ni mide impacto causal. Solo informa cuánto mejora una partición en la muestra y el nodo actuales.

12.1.3. Índice de Gini

El índice de Gini para clasificación se define como

$$G(R) = 1 - \sum_{k=1}^K p_k^2 = \sum_{k=1}^K p_k(1 - p_k).$$

Puede interpretarse como la probabilidad de etiquetar incorrectamente una observación si se asigna una clase al azar según la distribución del nodo. En clasificación binaria, $G(p) = 2p(1 - p)$, con máximo 0.5 en $p = 0.5$ y mínimo cero en nodos puros. La reducción es

$$\Delta G = G(R) - \sum_r \frac{n_r}{n} G(R_r).$$

Para el ejemplo anterior, $G(R) = 1 - (0.4^2 + 0.6^2) = 0.48$. Las impurezas hijas son 0.32 y aproximadamente 0.353, de modo que $\Delta G \approx 0.137$. El valor no es

comparable numéricamente con la ganancia entrópica porque las escalas difieren, pero ambos criterios suelen ordenar de manera parecida las divisiones.

Entropía penaliza con algo más de intensidad los cambios cerca de probabilidades extremas; Gini evita logaritmos y es computacionalmente simple. En la práctica, la elección entre ambos suele importar menos que profundidad, tamaño mínimo de hoja, calidad de atributos y validación. Si cambiar Gini por entropía transforma radicalmente el desempeño, conviene examinar inestabilidad, tamaño muestral y reglas seleccionadas, no declarar automáticamente superior a uno.

La llamada *importancia Gini* de un atributo suma las reducciones ponderadas de impureza obtenidas por sus divisiones. No debe confundirse con el índice de una sola región ni con la importancia por permutación. Está sesgada hacia variables continuas o con muchos cortes posibles y puede repartir o concentrar importancia arbitrariamente entre predictores correlacionados. Es una descripción interna del árbol, no evidencia de que intervenir sobre ese atributo cambie el resultado.

12.1.4. Árboles de clasificación y regresión

Los árboles de clasificación optimizan pureza y producen clases o probabilidades. Si el costo de confundir una pieza crítica con una conforme es mayor que el error inverso, puede introducirse una matriz de costos o decidir con probabilidades y umbrales posteriores. La clase mayoritaria no siempre representa la acción óptima. Para costos $C(a, k)$, la acción recomendada en una hoja es

$$a^* = \arg \min_a \sum_k C(a, k) \hat{p}(k | R_m).$$

Los árboles de regresión suelen elegir la división que minimiza la suma de errores cuadrados:

$$(j^*, s^*) = \arg \min_{j, s} \left[\sum_{i: x_i \in R_{\text{izq}}} (y_i - \bar{y}_{\text{izq}})^2 + \sum_{i: x_i \in R_{\text{der}}} (y_i - \bar{y}_{\text{der}})^2 \right].$$

La reducción equivale a la variabilidad del padre menos la variabilidad ponderada de las hijas. Con pérdida absoluta, la constante óptima sería la mediana; con una pérdida de cuantiles pueden estimarse percentiles condicionales útiles para inventarios de energía o tiempos de viaje.

Considérese la duración de una operación industrial: para temperaturas menores o iguales a 70 grados, los tiempos son 8, 9 y 10 minutos; por encima, 14, 15 y 16. Sin división, la media es 12 y la suma de cuadrados es 46. Con el corte, las medias son 9 y 15, y la suma residual es 4. La reducción de 42 justifica la partición en la muestra. Aun así, el salto exacto en 70 puede ser un artefacto; un fenómeno físico suave se aproxima mediante escalones.

Las ventajas comunes son: captura automática de interacciones, invariancia ante transformaciones monótonas del orden de una variable y poca necesidad de escalado. Las limitaciones incluyen discontinuidad, baja precisión para relaciones suaves, mala extrapolación e inestabilidad. En clasificación con fuerte desbalance, la exactitud y la pureza pueden ocultar una sensibilidad insuficiente; deben revisarse métricas por clase, calibración y costos.

12.1.5. Profundidad y complejidad

Un árbol profundo tiene baja restricción estructural: puede aislar observaciones y alcanzar error de entrenamiento casi nulo. Esa capacidad reduce sesgo, pero aumenta varianza. Un árbol poco profundo es más estable e interpretable, aunque puede omitir interacciones relevantes. La profundidad máxima, el mínimo de observaciones para dividir, el mínimo por hoja, la reducción mínima de impureza y el máximo de hojas son formas de *pre-poda*.

La complejidad efectiva no depende solo de la profundidad. Un árbol binario completo de profundidad d puede tener hasta 2^d hojas, pero uno desbalanceado puede tener muchas menos. Hojas con pocos casos ofrecen medias o proporciones de alta varianza. Si una hoja binaria contiene n_m observaciones y la probabilidad estimada es $\hat{p}_m$, su error estándar aproximado es $\sqrt{\hat{p}_m(1 - \hat{p}_m)/n_m}$; una predicción de 0.9 basada en diez casos es mucho menos precisa que la misma proporción basada en mil.

Las curvas de aprendizaje ayudan a diagnosticar. Error bajo en entrenamiento y alto en validación sugiere exceso de complejidad; errores altos y cercanos sugieren sesgo, variables insuficientes o señal débil. Debe observarse el desempeño medio y su dispersión entre particiones. Una profundidad elegida por una única división entrenamiento/prueba puede responder al azar de esa división.

La selección correcta anida el ajuste dentro de validación: para cada pliegue se construyen todas las transformaciones usando solo el entrenamiento, se ajustan complejidades candidatas y se evalúa en datos no vistos. Si se exploran muchas configuraciones y se reporta la mejor sobre el mismo conjunto de prueba, ese conjunto deja de ser una evaluación independiente. En datos temporales o agrupados por planta, conductor o cliente, los pliegues deben respetar la estructura para evitar que observaciones relacionadas aparezcan a ambos lados.

12.1.6. Poda

La poda posterior permite crecer un árbol amplio y retirar ramas cuya contribución no compensa su complejidad. En la poda por costo-complejidad se define, para un subárbol T ,

$$R_\alpha(T) = R(T) + \alpha|T|,$$

donde $R(T)$ es el error o impureza total de las hojas, $|T|$ su número de hojas y $\alpha \geq 0$ el precio de cada región terminal. Cuando $\alpha = 0$ se favorece el ajuste; al aumentarlo, estructuras más pequeñas pueden minimizar el objetivo.

Para un nodo interno t , sea T_t la rama que nace en él. Comparar reemplazar toda la rama por una hoja conduce al valor

$$g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}.$$

Este cociente es el aumento de error por hoja eliminada. La estrategia de *enlace más débil* poda primero el nodo con menor $g(t)$ y genera una secuencia anidada de subárboles. Validación cruzada selecciona α . La regla de un error estándar puede

preferir el árbol más simple cuyo error esté a una desviación estándar del mínimo, favoreciendo estabilidad e interpretación.

Pseudocódigo: poda costo-complejidad

```
PODAR(árbol grande T, datos de validación):  
    secuencia <- [T]  
    mientras T tenga nodos internos:  
        calcular g(t) para cada nodo interno  
        eliminar la rama con menor g(t), reemplazándola por una hoja  
        agregar el nuevo T a secuencia  
    evaluar cada subárbol con validación apropiada  
    elegir el subárbol según error, incertidumbre y simplicidad  
    devolver subárbol elegido
```

La poda no repara fuga de información, etiquetas defectuosas ni una partición de evaluación inadecuada. Tampoco garantiza que reglas cercanas al nodo raíz sean estables. Conviene repetir el ajuste en remuestras y registrar con qué frecuencia aparecen atributos y umbrales similares. Si la finalidad es comunicar reglas operativas, una pequeña pérdida predictiva puede ser aceptable; esa decisión debe explicitarse, no ocultarse bajo una métrica agregada.

12.1.7. Interpretación de reglas

Cada camino desde la raíz hasta una hoja es una conjunción de condiciones. Por ejemplo:

Si vibración RMS > 4.2, temperatura > 78 y antigüedad del componente > 18 meses, entonces riesgo estimado de defecto crítico = 0.73.

La regla describe una región de asociación predictiva. No afirma que elevar la temperatura cause el defecto, porque temperatura puede representar carga, turno o tipo de máquina. Para interpretar responsablemente una hoja deben comunicarse la predicción, el número de casos, la distribución de clases, el periodo de referencia y el comportamiento en validación. Un porcentaje sin denominador induce falsa precisión.

Las reglas también requieren verificar cobertura y consistencia. Cobertura es la proporción de observaciones que cumple la regla; precisión o confianza es la proporción de resultados objetivo dentro de ella. Una regla con 100% de precisión y 0.1% de cobertura puede ser irrelevante o espuria. Dos hojas pueden recomendar la misma acción y simplificarse conceptualmente, aunque el árbol las mantenga separadas.

Los umbrales son sensibles a medición. Una diferencia entre 4.19 y 4.21 no necesariamente justifica acciones opuestas si el sensor tiene error de ± 0.1 . Deben realizarse análisis de sensibilidad alrededor de cortes, revisar muestras próximas y, cuando proceda, definir una zona de abstención. Restricciones monotónicas o revisión experta pueden ser necesarias si las reglas contradicen conocimiento físico, pero nunca deben imponerse para maquillar datos problemáticos.

Una lectura disciplinada sigue cuatro preguntas: ¿qué población llega a la hoja?, ¿qué evidencia sostiene la estimación?, ¿se reproduce fuera de muestra?, ¿qué decisión y costo están asociados? La legibilidad de un árbol pequeño es una propiedad de representación; no equivale automáticamente a transparencia

del sistema completo, que incluye selección de datos, variables, umbrales y procedimientos de actualización.

12.1.8. Ejemplo práctico guiado: árbol para priorización de incidentes

Una empresa manufacturera debe priorizar incidentes de maquinaria para inspección en las próximas dos horas. La unidad de análisis es un incidente registrado; la etiqueta vale 1 si, después de una revisión independiente, se confirmó riesgo de parada crítica. Se dispone de vibración, temperatura, variación de corriente, edad del componente, tipo de máquina y carga. El costo de omitir un incidente crítico supera al de inspeccionar uno no crítico.

Paso 1: formulación y partición. Se fija como tiempo de predicción el instante de alerta. Se excluyen variables creadas después de la inspección, como código final de avería. Se separan periodos: meses anteriores para desarrollo y un mes posterior para prueba. Incidentes de una misma secuencia operativa no se reparten entre pliegues. Se comparará contra reglas actuales y contra un clasificador que siempre predice la prevalencia.

Paso 2: crecimiento. En un nodo con 200 incidentes, 50 críticos, $G = 2(0.25)(0.75) = 0.375$. Un corte de vibración produce una hija alta con 60 casos y 36 críticos, $G_A = 0.48$, y una baja con 140 casos y 14 críticos, $G_B = 0.18$. La impureza posterior es $0.3(0.48) + 0.7(0.18) = 0.27$ y la reducción, 0.105. Aunque la rama alta no es pura, concentra riesgo y mejora la decisión.

Paso 3: control y poda. Se crece una secuencia de árboles restringiendo hojas diminutas. Validación temporal compara sensibilidad, precisión, área precisión-recall, costo esperado y calibración. El árbol de mínimo error tiene 18 hojas; otro de 9 queda dentro de un error estándar y muestra menor variabilidad entre meses. Se elige este último si satisface la sensibilidad mínima.

Paso 4: decisión. Las probabilidades no se convierten automáticamente en prioridad. Para costo de falso negativo C_{FN} y falso positivo C_{FP} , con costos constantes, el umbral teórico es $C_{FP}/(C_{FP} + C_{FN})$, sujeto a capacidad de inspección y calibración. Si solo pueden atenderse 20 alertas por turno, también se evalúa precisión entre las 20 primeras.

Paso 5: diagnóstico. Se revisan hojas por planta y tipo de máquina, estabilidad de umbrales, errores cerca de cortes y deriva mensual. Si temperatura domina pero su sensor cambió, el aparente progreso puede no ser transportable. Se documentan falsos negativos críticos y condiciones de abstención ante valores fuera de rango.

El resultado no es solo un diagrama: es un árbol podado, un umbral operativo, intervalos de desempeño, reglas de monitoreo y un registro de limitaciones. La claridad se evalúa junto con generalización, no en sustitución de ella.

12.2. Máquinas de soporte vectorial

Las máquinas de soporte vectorial (SVM) construyen fronteras usando geometría y regularización. En su forma lineal buscan un hiperplano que separe clases con el mayor margen posible. En su forma kernel representan fronteras no lineales mediante productos internos en un espacio de características, sin calcular explícitamente todas sus coordenadas. Son especialmente útiles en espacios de dimen-

sión alta y muestras medianas, pero exigen escalado, selección cuidadosa de hiperparámetros y atención al costo computacional.

12.2.1. Hiperplanos y fronteras de decisión

Para clasificación binaria se codifica $y_i \in \{-1, +1\}$. Un hiperplano es el conjunto

$$\{x : w^T x + b = 0\},$$

donde w es normal a la superficie y b desplaza la frontera. La función de decisión es $f(x) = w^T x + b$ y la clase predicha, $\text{sign}(f(x))$. La magnitud de f no es una probabilidad: es una puntuación cuya escala depende de w .

La distancia perpendicular de un punto x al hiperplano se deriva proyectando sobre el vector normal:

$$d(x, H) = \frac{|w^T x + b|}{\|w\|_2}.$$

Si $y_i(w^T x_i + b) > 0$, el punto está correctamente clasificado. El *margen funcional* $y_i f(x_i)$ cambia si w y b se multiplican por una constante; el margen geométrico $y_i f(x_i)/\|w\|$ no cambia. Esta distinción permite fijar una escala conveniente al formular la optimización.

En dos dimensiones, el hiperplano es una recta; en tres, un plano; en p dimensiones conserva la misma definición. Muchas fronteras pueden separar perfectamente una muestra. Una que pase muy cerca de observaciones es vulnerable a pequeñas perturbaciones. SVM selecciona la de mayor separación geométrica bajo una penalización determinada.

Para más de dos clases suelen combinarse clasificadores binarios: uno contra el resto entrena K modelos; uno contra uno entrena $K(K-1)/2$ y agrega votos o puntuaciones. La evaluación debe incluir confusiones por clase. Un esquema multiclase puede tener buen promedio y fallar sistemáticamente en una categoría minoritaria de reclamaciones.

12.2.2. Margen máximo

Si los datos son linealmente separables, puede elegirse la escala de modo que los puntos más cercanos satisfagan $y_i(w^T x_i + b) = 1$. Los planos de soporte son $w^T x + b = 1$ y $w^T x + b = -1$; la distancia entre ellos es $2/\|w\|$. Maximizarla equivale a minimizar $\|w\|^2/2$:

$$\min_{w, b} \frac{1}{2} \|w\|_2^2 \quad \text{sueto a} \quad y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n.$$

Es un problema convexo: cualquier óptimo local es global. La norma actúa como regularizador, pues una norma pequeña produce una función menos sensible a perturbaciones. Sin embargo, separabilidad perfecta puede deberse a alta dimensión, error de medición o fuga, no a una estructura generalizable.

La formulación dual introduce multiplicadores $\alpha_i \geq 0$ y conduce a

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j,$$

sujeto a $\sum_i \alpha_i y_i = 0$. La solución cumple $w = \sum_i \alpha_i y_i x_i$. Por tanto, los datos aparecen solo mediante productos internos y los puntos con $\alpha_i > 0$ determinan la frontera. Esta propiedad dará lugar al truco kernel.

El margen máximo no optimiza directamente exactitud, AUC o costo comercial. Optimiza una función sustituta con propiedades matemáticas favorables. Si los errores tienen costos asimétricos, pueden usarse penalizaciones de clase o ajustar el umbral de la puntuación en validación. Además, para reportar probabilidades se necesita calibración posterior con datos que no hayan sido utilizados para ajustar la frontera.

12.2.3. Vectores de soporte

Los vectores de soporte son observaciones con multiplicadores duales positivos. En el caso separable y en posición general se encuentran sobre los planos del margen. Los puntos alejados de la frontera tienen $\alpha_i = 0$ y no modifican la solución mientras permanezcan correctamente ubicados. La predicción puede escribirse como

$$f(x) = \sum_{i \in SV} \alpha_i y_i x_i^T x + b.$$

Esto muestra que SVM es una expansión basada en ejemplos críticos. No significa que todos esos ejemplos sean prototipos típicos: suelen ser precisamente los más ambiguos. Una alta proporción de vectores de soporte puede indicar solapamiento, ruido, regularización fuerte o una representación inadecuada. También aumenta el costo de predicción en modelos kernel, porque debe evaluarse la similitud con cada soporte.

Las condiciones de Karush-Kuhn-Tucker conectan geometría y optimización. En margen duro, la complementariedad exige

$$\alpha_i [y_i (w^T x_i + b) - 1] = 0.$$

Si una restricción es estricta, el multiplicador es cero; si influye, la observación toca el margen. En margen blando aparecen casos dentro del margen o mal clasificados con multiplicadores limitados por C .

Un diagnóstico útil consiste en revisar qué tipos de observaciones se convierten repetidamente en soporte a través de pliegues. Si predominan registros duplicados, errores de etiqueta o una planta con sensores descalibrados, la frontera puede estar modelando problemas de calidad. Eliminar puntos solo porque son soportes sería incorrecto: debe investigarse su procedencia. La robustez se comprueba perturbando mediciones dentro de tolerancias plausibles y reentrenando, no interpretando la lista como anomalías automáticas.

12.2.4. Margen blando

Los datos reales rara vez son separables sin errores. El margen blando incorpora variables de holgura $\xi_i \geq 0$:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i.$$

Si $\xi_i = 0$, el punto está fuera o sobre el margen correcto; si $0 < \xi_i \leq 1$, está bien clasificado pero dentro del margen; si $\xi_i > 1$, está mal clasificado. Para w, b fijos, la menor holgura es $\max(0, 1 - y_i f(x_i))$. Sustituyendo se obtiene la pérdida bisagra:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i f(x_i)).$$

La pérdida es cero solo cuando el margen funcional alcanza uno; por ello no solo penaliza errores, sino también aciertos demasiado próximos a la frontera. Frente a la pérdida 0-1, es convexa y optimizable.

C controla el compromiso. Un C grande castiga fuertemente violaciones, tiende a un margen estrecho y puede ajustarse al ruido. Un C pequeño tolera violaciones, ensancha el margen y aumenta regularización. La escala depende de la convención: algunas formulaciones promedian la pérdida y usan un parámetro inverso. Debe interpretarse la definición concreta, no memorizar que un número tiene igual efecto en todo sistema.

Con desbalance, asignar el mismo costo a todos los ejemplos puede favorecer la clase mayoritaria. Una extensión usa C_i por clase u observación. Esto modifica el objetivo, pero no reemplaza la evaluación con precisión-recall, sensibilidad, especificidad, calibración y costo esperado. Una ponderación estimada usando la prueba constituye fuga; debe fijarse con entrenamiento y conocimiento operativo.

12.2.5. Funciones kernel

Una transformación $\phi(x)$ puede convertir una relación no lineal en lineal en un espacio de características. El dual requiere solo productos $\phi(x_i)^T \phi(x_j)$. Un kernel válido calcula

$$K(x, z) = \langle \phi(x), \phi(z) \rangle$$

sin construir explícitamente ϕ . La función de decisión pasa a ser

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b.$$

Kernels frecuentes son:

$$K_{\text{lineal}}(x, z) = x^T z,$$

$$K_{\text{polinómico}}(x, z) = (\gamma x^T z + r)^d,$$

$$K_{\text{RBF}}(x, z) = \exp(-\gamma \|x - z\|^2).$$

El kernel lineal es apropiado cuando la dimensión es alta o se busca una referencia simple. El polinómico representa interacciones hasta cierto grado, pero puede ser sensible a escala y parámetros. RBF mide similitud local: con γ pequeño, la influencia de cada punto se extiende ampliamente y la frontera es suave; con γ grande, la influencia es estrecha y la frontera puede formar islas alrededor de observaciones.

No toda función de similitud es un kernel admisible. Una condición fundamental es que la matriz de Gram $K_{ij} = K(x_i, x_j)$ sea semidefinida positiva para cualquier conjunto finito, lo que preserva la interpretación como producto interno y la convexidad. En aplicaciones especializadas pueden definirse kernels para secuencias o grafos, siempre justificando sus propiedades.

El truco kernel no hace gratuita la alta dimensión. La matriz de Gram ocupa orden n^2 y entrenar puede ser costoso. Asimismo, una frontera flexible no garantiza que existan datos cercanos a cada predicción. Deben identificarse casos fuera del soporte de entrenamiento y evitar extrapolaciones confiadas.

12.2.6. Parámetros de regularización y kernel

En una SVM RBF, C y γ interactúan. C grande y γ grande permiten fronteras muy irregulares: bajo error de entrenamiento, muchos bolsillos locales y alta varianza. C pequeño y γ pequeño pueden producir subajuste. No es correcto ajustar uno y congelarlo antes de explorar el otro sin comprobar interacción.

La búsqueda suele realizarse en escala logarítmica porque cambios multiplicativos son más informativos: por ejemplo, una malla conceptual de $C \in \{10^{-3}, \dots, 10^3\}$ y $\gamma \in \{10^{-4}, \dots, 10^2\}$. Los valores concretos dependen del escalado y tamaño. Una búsqueda aleatoria puede cubrir mejor espacios amplios; una estrategia sucesiva asigna más presupuesto a candidatos prometedores. Cualquier estrategia debe vivir dentro de los pliegues de validación.

El mapa de desempeño medio y dispersión es más útil que una única combinación ganadora. Una meseta amplia indica robustez; un pico estrecho puede ser selección por ruido. Si varios candidatos son estadísticamente indistinguibles, se prefiere menor complejidad, menor costo o mejor estabilidad. Para clasificación financiera temporal, los pliegues deben simular meses futuros y no barajar indiscriminadamente transacciones.

La calibración es otro hiperproceso. Puntuaciones SVM pueden transformarse en probabilidades con una función logística o un ajuste isotónico. El calibrador debe entrenarse sobre predicciones fuera de muestra o un subconjunto separado; ajustarlo sobre las mismas puntuaciones con las que se entrenó SVM produce optimismo. Se evalúan Brier, pérdida logarítmica y diagramas de confiabilidad, además de discriminación.

Finalmente, el umbral es una decisión, no un hiperparámetro geométrico. Puede cambiarse según costos o capacidad sin reentrenar la SVM, siempre que se valide.

Mezclar selección de C , calibración y umbral sobre un único conjunto pequeño aumenta sobreajuste de selección.

12.2.7. Escalado y costo computacional

SVM se basa en productos y distancias. Si un atributo energético varía entre 0 y 100 000 y una proporción entre 0 y 1, el primero dominará $\|x - z\|^2$ aunque no sea más relevante. La estandarización $z_j = (x_j - \mu_j)/\sigma_j$ o un escalado robusto son pasos esenciales. Los parámetros se estiman solo con el pliegue de entrenamiento y luego se aplican a validación; calcularlos con todos los datos filtra información.

Variables categóricas codificadas generan otra geometría. Una codificación ordinal impone distancias quizá inexistentes; una codificación indicadora aumenta dimensión. Atributos ausentes e indicadores de ausencia deben tratarse dentro del mismo flujo. Outliers pueden distorsionar media y desviación; el escalado robusto y la investigación de calidad son alternativas, no la eliminación automática.

En SVM kernel, almacenar la matriz de similitudes requiere $O(n^2)$ memoria y la optimización puede acercarse a costos entre cuadráticos y cúbicos según datos y solucionador. La predicción cuesta aproximadamente $O(|SV|p)$ para RBF. Para millones de observaciones pueden ser preferibles SVM lineales, aproximaciones de kernel, muestreo cuidadosamente diseñado o ensambles escalables. Entrenar sobre una muestra sesgada para ahorrar costo altera el problema.

Los diagnósticos incluyen tiempo y memoria por pliegue, número y proporción de soportes, sensibilidad a escalado, estabilidad de métricas y latencia de predicción. También debe comprobarse el rango: RBF tiende a puntuaciones determinadas por similitudes pequeñas ante puntos muy lejanos. Un detector de novedad o una política de abstención puede ser necesario.

La comparación de costo debe ser integral: preparación, búsqueda, calibración, inferencia y mantenimiento. Un modelo que mejora mínimamente AUC pero multiplica latencia y dificulta actualización puede no ser la mejor decisión de ingeniería.

12.2.8. Ejemplo práctico guiado: clasificación no lineal con SVM

Una entidad financiera estudia reclamaciones de transacciones para decidir cuáles requieren revisión especializada. La etiqueta indica fraude confirmado, y los atributos disponibles al decidir incluyen importe normalizado respecto del historial, desviación geográfica, hora, velocidad de transacciones recientes y antigüedad de la cuenta. No se usa el resultado posterior de investigación.

Hipótesis geométrica. El riesgo puede aumentar cuando coinciden importe atípico y alta velocidad, pero no con cualquiera por separado. Una frontera lineal sirve como referencia; una RBF puede capturar esa interacción local. Se reserva el trimestre más reciente como prueba y se valida por bloques temporales.

Preparación. En cada pliegue se imputan valores, codifican categorías y escalan atributos usando solo el pasado. La búsqueda conjunta explora kernel lineal y RBF, C y γ . Se registran área precisión-recall, sensibilidad a capacidad fija de revisión, costo, calibración, proporción de soportes y tiempo.

Supóngase que los resultados medios son:

Modelo	PR-AUC	Sensibilidad en 200 revisiones	Soportes	Observación
Lineal, $C = 0.1$	0.41	0.62	38%	estable
RBF, $C = 10$, $\gamma = 0.5$	0.49	0.71	44%	meseta robusta
RBF, $C = 1000$, $\gamma = 20$	0.55	0.54	81%	sobreajuste
	entrenamiento / 0.38 validación			

El tercer modelo ilustra que una frontera visualmente detallada y un excelente ajuste interno no implican generalización. Se selecciona el segundo si mantiene su ventaja en la prueba temporal y su latencia es admisible. Después se calibra con predicciones fuera de pliegue y se determina un umbral acorde con la capacidad diaria.

Análisis de fallos. Se examinan falsos negativos por tipo de comercio y antigüedad, casos cercanos al umbral, registros fuera del rango histórico y deriva. Si el desempeño cae tras una campaña comercial, quizá cambió la población y no el algoritmo. Una auditoría comprueba que ningún atributo sea una consecuencia del proceso de investigación.

Conclusión. Comparar kernels implica más que escoger la métrica máxima: exige un baseline lineal, validación temporal, mapa de hiperparámetros, calibración y costo computacional. La SVM RBF se adopta solo si la mejora no lineal persiste bajo ese protocolo.

12.3. Métodos de ensamble

Un ensamble combina modelos base para obtener una predicción más precisa o estable. La mejora surge cuando los componentes cometen errores no perfectamente correlacionados y el mecanismo de combinación aprovecha esa diversidad. Bagging reduce varianza mediante paralelismo y remuestreo; Random Forest añade aleatoriedad de atributos; boosting construye modelos secuenciales que corrigen errores; stacking aprende cómo combinar modelos heterogéneos.

12.3.1. Combinación de modelos

Sean $\hat{f}_1(x), \dots, \hat{f}_B(x)$ predictores. En regresión, un promedio simple es

$$\bar{f}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

Si cada estimador tiene varianza σ^2 y correlación de error común ρ , entonces

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Al crecer B , desaparece el componente no correlacionado, pero permanece $\rho\sigma^2$. La diversidad es por ello tan importante como la calidad individual. Promediar copias casi idénticas aporta poco.

En clasificación pueden promediarse probabilidades o votar clases. Promediar probabilidades conserva información para ajustar umbrales, pero exige verificar calibración. El voto mayoritario puede ocultar que tres modelos tienen confianza apenas superior a 0.5. Con pesos $w_b \geq 0$, $\sum_b w_b = 1$, se obtiene $\hat{p} = \sum_b w_b \hat{p}_b$; estimar pesos sobre prueba causaría optimismo.

La descomposición sesgo-varianza explica parte del comportamiento. Bagging suele mantener sesgo y reducir varianza; boosting puede reducir ambos, aunque se vuelve sensible a hiperparámetros, ruido y pérdida. Ningún ensamble garantiza mejora: modelos base sesgados en la misma dirección pueden reforzarse; una fuga compartida produce una combinación convincentemente incorrecta.

La comparación justa requiere idéntica definición de objetivo, atributos disponibles, divisiones, métricas y presupuesto de búsqueda. También debe incluir un baseline simple. Si un ensamble gana solo porque recibió más ingeniería o más oportunidades de ajuste, no se ha aislado el valor del método.

12.3.2. Bagging

Bootstrap aggregating genera B muestras bootstrap de tamaño n mediante selección con reemplazo, ajusta un modelo en cada una y agrega predicciones. La probabilidad de que una observación no aparezca en una remuestra es $(1 - 1/n)^n \approx e^{-1} \approx 0.368$. Así, cada modelo utiliza alrededor de 63.2% de observaciones únicas y deja otras *out-of-bag* (OOB).

Pseudocódigo: bagging

```
BAGGING(D, B, algoritmo_base):  
    modelos <- lista vacía  
    para b = 1 hasta B:  
        D_b <- muestra con reemplazo de D, de tamaño |D|  
        modelo_b <- ajustar algoritmo_base sobre D_b  
        agregar modelo_b a modelos  
    para predecir x:  
        agregar las B predicciones por media o voto
```

Para estimar error OOB, cada observación se predice solo con modelos que no la incluyeron. Esto ofrece una evaluación interna eficiente y permite estudiar convergencia al aumentar B . No reemplaza una prueba externa cuando existen tiempo, grupos, deriva o múltiples decisiones de diseño. El bootstrap ordinario rompe dependencias temporales y puede ser inadecuado para series o eventos agrupados.

Bagging funciona especialmente bien con aprendices de alta varianza, como árboles profundos. Aumentar B no suele causar sobreajuste clásico por sí solo: el promedio converge, aunque costo y memoria crecen. El beneficio se estabiliza, por lo que debe observarse la curva OOB frente al número de modelos.

Si las muestras son muy similares y un predictor fuerte domina siempre el árbol, los modelos permanecen correlacionados. Random Forest aborda este problema restringiendo candidatos por nodo. En datos desbalanceados pueden diseñarse

remuestras estratificadas, pero alterar prevalencias afecta probabilidades; se requiere corrección o calibración sobre la distribución objetivo.

12.3.3. Random Forest

Random Forest combina bootstrap con selección aleatoria de atributos. En cada nodo se considera un subconjunto de m_{try} variables en lugar de las p disponibles. Un predictor dominante no puede participar en todas las divisiones, lo que aumenta diversidad entre árboles y puede reducir la correlación del ensamble.

Pseudocódigo: Random Forest

```
BOSQUE_ALEATORIO(D, B, m_try):  
  para b = 1 hasta B:  
    D_b <- remuestra bootstrap de D  
    crecer un árbol amplio sobre D_b:  
      en cada nodo seleccionar al azar m_try atributos  
      elegir la mejor división solo entre esos atributos  
      continuar hasta cumplir restricciones de hoja  
  agregar predicciones de todos los árboles
```

m_{try} pequeño aumenta diversidad pero puede elevar sesgo; grande aproxima bagging de árboles. El mínimo de hoja controla suavidad: hojas mayores estabilizan probabilidades y regresión. El número de árboles se aumenta hasta que error y métricas se estabilicen. La profundidad, remuestreo y pesos de clase completan el ajuste.

La predicción OOB permite error e importancia por permutación sin un conjunto adicional, bajo supuestos de independencia. Aun así, datos agrupados requieren remuestreo por grupo. La importancia por disminución de impureza favorece atributos con numerosos cortes. La permutación es preferible para medir dependencia predictiva, aunque también se complica con correlación.

Random Forest es robusto y suele ser un baseline tabular fuerte: modela interacciones, requiere poco preprocesamiento geométrico y paraleliza. Sus desventajas son tamaño, latencia, menor interpretabilidad directa y escasa extrapolación. En regresión energética, una predicción forestal es un promedio de valores de hojas observados; no prolongará automáticamente una tendencia física fuera del rango.

Un bosque no corrige etiquetas contaminadas ni cambio de concepto. Si el OOB es excelente pero la prueba futura falla, deben investigarse estructura temporal, variables posteriores al evento y deriva antes de ajustar más árboles.

12.3.4. Boosting

Boosting construye una suma secuencial de aprendices débiles. En AdaBoost binario, cada observación tiene un peso w_i . Al inicio son iguales. En la iteración m se ajusta un clasificador $h_m(x) \in \{-1, +1\}$, se calcula su error ponderado

$$e_m = \frac{\sum_i w_i \mathbf{1}[y_i \neq h_m(x_i)]}{\sum_i w_i},$$

y se asigna

$$\alpha_m = \frac{1}{2} \log \frac{1 - e_m}{e_m}.$$

Los pesos se actualizan como $w_i \leftarrow w_i \exp[-\alpha_m y_i h_m(x_i)]$ y se normalizan. Casos mal clasificados aumentan peso. La predicción final es $\text{sign}(\sum_m \alpha_m h_m(x))$.

Pseudocódigo: boosting adaptativo

```

inicializar pesos uniformes sobre observaciones
para m = 1 hasta M:
    ajustar aprendiz débil h_m usando los pesos actuales
    calcular error ponderado e_m
    si h_m no supera el azar, detener o revisar
    calcular peso del modelo alpha_m
    aumentar peso de errores y reducir peso de aciertos
    normalizar pesos
devolver suma ponderada de aprendices

```

AdaBoost puede verse como minimización progresiva de pérdida exponencial $\sum_i \exp[-y_i F(x_i)]$. Esta pérdida enfatiza observaciones con margen negativo. La concentración ayuda a corregir casos difíciles, pero también puede perseguir etiquetas erróneas y outliers. Diagnosticar qué registros acumulan peso es esencial.

Los aprendices suelen ser árboles pequeños. Tocones de una división representan efectos aditivos; profundidades mayores permiten interacciones. El número de iteraciones y complejidad base regulan capacidad. A diferencia de bagging, los modelos no son independientes y no se entrenan todos en paralelo. Boosting también puede operar con pérdidas distintas mediante Gradient Boosting.

12.3.5. Gradient Boosting

Gradient Boosting construye una función aditiva

$$F_M(x) = F_0(x) + \sum_{m=1}^M \nu \rho_m h_m(x),$$

donde h_m suele ser un árbol, ρ_m un tamaño de paso y $0 < \nu \leq 1$ la tasa de aprendizaje. La idea es descenso de gradiente en el espacio de funciones. En la iteración m se calculan los seudoresiduos

$$r_{im} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}},$$

se ajusta h_m para predecirlos y se actualiza F .

Con pérdida cuadrática, $L = (y - F)^2/2$, el gradiente negativo es $y_i - F(x_i)$: residuos ordinarios. Para clasificación logística, los residuos se relacionan con $y_i - p_i$, de modo que cada etapa corrige discrepancias probabilísticas.

Pseudocódigo: Gradient Boosting

```

elegir  $F_0$  que minimice la pérdida constante
para  $m = 1$  hasta  $M$ :
    calcular gradiente negativo de la pérdida para cada caso
    ajustar árbol  $h_m$  a esos seudoresiduos
    encontrar tamaño de paso  $\rho_m$ , si corresponde
     $F_m \leftarrow F_{(m-1)} + \text{tasa\_aprendizaje} * \rho_m * h_m$ 
devolver  $F_M$ 

```

Una tasa pequeña suele requerir más árboles y puede generalizar mejor. Profundidad controla orden de interacción; submuestrear filas o columnas introduce aleatoriedad y regularización. Restricciones L1/L2, mínimo de hoja y penalización de nuevas divisiones son comunes en implementaciones optimizadas, pero el principio sigue siendo minimizar una pérdida regularizada de forma secuencial.

El monitoreo de pérdida de validación frente a iteraciones permite parada temprana. La iteración de parada debe decidirse sin mirar prueba. Fallos frecuentes son árboles demasiado profundos, búsqueda extensa sobre un único pliegue, probabilidades no calibradas, categorías codificadas con información futura y optimización de una métrica desconectada del costo. En ruido de etiquetas, pérdidas robustas o revisión de datos pueden ser más valiosas que más iteraciones.

12.3.6. Stacking

Stacking aprende un metamodelo que combina predicciones de modelos base heterogéneos. Si $f_1, \dots, f_L$ son modelos de nivel 0, el nivel 1 recibe $z_i = (\hat{f}_1(x_i), \dots, \hat{f}_L(x_i))$ y aprende $g(z_i)$. La predicción final es $g(f_1(x), \dots, f_L(x))$.

El riesgo central es fuga. Si z_i se genera con un modelo entrenado usando (x_i, y_i) , el metamodelo ve predicciones demasiado optimistas. Deben usarse predicciones *out-of-fold* (OOF): cada fila es predicha por un modelo que no la vio.

Pseudocódigo: stacking sin fuga

```

dividir desarrollo en K pliegues
para cada modelo base l:
    para cada pliegue k:
        ajustar modelo l con los otros K-1 pliegues
        predecir el pliegue k y guardar predicciones OOF
ajustar metamodelo g con todas las predicciones OOF y etiquetas
reajustar cada modelo base sobre todo desarrollo
para un caso nuevo:
    obtener predicciones de los modelos reajustados
    pasarlas al metamodelo g

```

Un metamodelo lineal regularizado es a menudo suficiente y fácil de auditar. Pesos no negativos y suma uno pueden reducir extrapolaciones. Un metamodelo muy flexible puede sobreajustar un conjunto OOF pequeño. Añadir atributos originales al nivel 1 aumenta capacidad y riesgo.

La diversidad debe ser real: combinar cinco configuraciones casi idénticas de boosting quizá aporte menos que combinar modelo lineal, bosque y SVM con errores complementarios. Se estudian correlaciones de residuos y rendimiento por subgrupo. Stacking puede aprender a ignorar un modelo, lo cual no es un fallo.

La evaluación externa repite el proceso completo. Preprocesamiento, calibración

y ajuste de cada base ocurren dentro de sus pliegues. Este costo hace que stacking solo se justifique si la mejora es estable y operativamente relevante. También complica despliegue: todos los modelos, versiones y latencias pasan a ser dependencias.

12.3.7. Ajuste de hiperparámetros

Los ensambles poseen hiperparámetros interdependientes. En Random Forest destacan número de árboles, m_{try} , profundidad y mínimo de hoja. En boosting: iteraciones, tasa de aprendizaje, profundidad, submuestreo y regularización. En stacking: integrantes, generación OOF y complejidad del metamodelo. Ajustarlos sin un protocolo predefinido produce una competencia de sobreajuste.

Un proceso recomendable es:

1. Definir unidad, horizonte, métrica primaria, métricas de seguridad y presupuesto.
2. Reservar una prueba final que reproduzca despliegue.
3. Construir un baseline simple y una configuración razonable por familia.
4. Realizar búsqueda dentro de validación, con transformaciones internas a cada pliegue.
5. Comparar media, dispersión, costo y curvas de aprendizaje, no solo el máximo.
6. Seleccionar la configuración más simple dentro de una región competitiva.
7. Reajustar con desarrollo completo y evaluar una sola vez en prueba.

La optimización multiobjetivo reconoce que precisión, memoria, latencia, energía, equidad y explicabilidad pueden competir. El frente de Pareto contiene modelos para los que ningún objetivo mejora sin empeorar otro. Elegir en ese frente requiere prioridades institucionales, no una fórmula puramente estadística.

La incertidumbre de la comparación importa. Diferencias pequeñas entre métricas calculadas sobre los mismos casos deben analizarse de forma pareada y con intervalos adecuados. La selección entre cientos de candidatos aumenta el sesgo del ganador. Una validación externa o anidada reduce este efecto.

Diagnósticos esenciales son: brecha entrenamiento-validación, variabilidad entre pliegues y semillas, evolución con número de árboles, calibración, matrices de error por segmento, latencia y sensibilidad a deriva. Si el ranking cambia con cada semilla, no existe evidencia robusta de superioridad.

12.3.8. Ejemplo práctico guiado: competencia controlada de modelos

Una red de movilidad urbana predice si un trayecto programado llegará con más de diez minutos de demora. La decisión se toma 30 minutos antes de la salida y habilita reasignación preventiva. Se comparan un árbol podado, Random Forest y Gradient Boosting. Las variables incluyen franja horaria, zona, duración planificada, congestión reciente disponible y características del vehículo.

Protocolo común. Se usa expansión temporal: entrenar en semanas 1-8 y validar en 9; luego 1-9 y validar en 10, y así sucesivamente. La semana 13 queda como prueba. Todos reciben la misma información al mismo tiempo de corte. Se evalúan PR-AUC, sensibilidad a un máximo de 100 intervenciones diarias, Brier, latencia y estabilidad por zona.

Modelo	PR-AUC validación	Sensibilidad@100	Brier	Latencia relativa
Árbol podado	0.36 ± 0.03	0.51	0.171	1
Random Forest	0.44 ± 0.02	0.63	0.154	7
Gradient Boosting	0.46 ± 0.04	0.65	0.149	4

Boosting tiene mejor media, pero su ventaja sobre el bosque es pequeña y más variable. Se revisan curvas por semana: una caída coincide con un cambio de rutas. El bosque resulta algo más estable en zonas con pocos trayectos. La elección dependerá de si la mejora de dos puntos en sensibilidad compensa la variabilidad y el mantenimiento.

Controles. Se compara contra prevalencia y regla operativa. Las transformaciones se reajustan en cada ventana. Se repiten semillas, se calibran candidatos usando predicciones temporales OOF y el umbral se fija por capacidad. La prueba final se abre después de congelar todo el flujo.

Análisis de errores. Se inspeccionan falsos negativos en eventos especiales, diferencias por zona y vehículos nuevos. Si el modelo usa congestión registrada después del tiempo de decisión, se elimina aunque eleve métricas. Si las zonas periféricas muestran peor sensibilidad por menor cobertura de sensores, se documenta y se considera abstención o inversión en datos.

Una competencia controlada no busca coronar un algoritmo universal. Busca estimar qué sistema satisface mejor la especificación bajo incertidumbre y recursos reales. El árbol conserva valor como referencia interpretable incluso si no se despliega.

12.4. Explicabilidad y aprendizaje distribuido

La explicabilidad estudia cómo describir el comportamiento de modelos y predicciones. Puede ser intrínseca, como las reglas de un árbol pequeño, o posterior al ajuste, como permutación, dependencia parcial y SHAP. Estas herramientas responden preguntas distintas y dependen de una población de referencia. El aprendizaje federado, por su parte, permite colaborar sin centralizar datos brutos, pero introduce heterogeneidad, comunicación y nuevas superficies de riesgo.

12.4.1. Interpretabilidad global y local

La interpretación global pregunta cómo se comporta el modelo en una población: qué variables utiliza, qué formas promedio aprende, qué interacciones existen y dónde falla. La local pregunta por qué produjo una puntuación para un caso concreto. Una tercera escala, la interpretación por cohortes, estudia subpoblaciones como plantas, regiones o periodos. Confundir escalas genera contradicciones aparentes: un atributo puede ser importante globalmente y casi irrelevante para un individuo.

También debe distinguirse entre explicación del modelo y explicación del fenómeno. Una explicación fiel describe $\hat{f}$; no demuestra el mecanismo que genera Y . Si el modelo usa consumo energético nocturno para predecir averías, una atribución alta indica dependencia predictiva bajo los datos observados. No prueba que reducir consumo evite averías. Para afirmaciones causales se necesitan supuestos, diseño e identificación específicos.

Las propiedades deseables incluyen fidelidad, estabilidad, alcance, comprensibilidad y utilidad. Pueden entrar en tensión: una regla local simple es comprensible pero solo aproximada; una descomposición exacta puede depender de una referencia poco intuitiva. La explicación debe acompañarse de predicción, incertidumbre, población de referencia y advertencias.

Un protocolo de interpretación comienza por una pregunta: ¿auditar fuga?, ¿comunicar una decisión?, ¿descubrir fallos?, ¿verificar una restricción física? Luego selecciona herramienta y conjunto de referencia, contrasta más de un método, realiza análisis de sensibilidad y valida con conocimiento del dominio. Una visualización atractiva sin pregunta definida favorece narrativas posteriores.

La interpretabilidad no compensa un modelo inválido. Primero se verifica generalización, calibración, subgrupos y deriva. Después se explica. Una explicación estable de una predicción sistemáticamente errónea sigue siendo un fracaso.

12.4.2. Importancia por permutación

La importancia por permutación mide cuánto empeora una métrica cuando se rompe la asociación entre un atributo y el resultado, manteniendo fijo el modelo. Sobre un conjunto de evaluación D_{eval} , sea $S(D, f)$ una puntuación donde mayor es mejor. Para el atributo j :

$$I_j = S(D_{eval}, f) - S(D_{eval}^{\pi_j}, f),$$

donde D^{π_j} permuta los valores de x_j entre filas. Se repite la permutación para obtener media y dispersión. Si se usa una pérdida donde menor es mejor, se invierte la resta.

Pseudocódigo: importancia por permutación

```

calcular desempeño base del modelo en evaluación
para cada atributo j:
    para r = 1 hasta R:
        copiar evaluación
        permutar solo la columna j
        calcular deterioro respecto del desempeño base
    reportar media e intervalo de los R deterioros

```

La medición debe hacerse fuera del entrenamiento. Puede calcularse con la métrica operativa y por cohortes. Una importancia cercana a cero significa que permutar esa variable no degrada el modelo en esa población; no significa ausencia de relación real. Una importancia negativa puede aparecer por azar, sobreajuste o porque romper una señal perjudicial mejora desempeño.

La correlación es el principal desafío. Si dos sensores contienen información redundante, permutar uno deja disponible el otro y ambos parecen poco importantes. A la inversa, permutar una variable genera combinaciones irreales, exagerando el deterioro. La permutación condicional intenta reorganizar dentro de grupos similares o según una distribución condicional, pero cambia la pregunta y requiere estimación adicional.

La importancia depende de modelo, datos, métrica y momento. No es un ranking universal ni tiene dirección de efecto. Deben reportarse incertidumbre y grupos

correlacionados, contrastarse con ablation o conocimiento experto y evitar frases causales como “el atributo produce el resultado”.

12.4.3. Gráficos de dependencia parcial

La dependencia parcial del atributo X_j se define como

$$PD_j(z) = \mathbb{E}_{X_{-j}}[\hat{f}(z, X_{-j})] \approx \frac{1}{n} \sum_{i=1}^n \hat{f}(z, x_{i,-j}).$$

Para cada valor z , se reemplaza x_{ij} de todas las filas por z , se predice y se promedia. El gráfico muestra el efecto promedio del modelo al variar X_j sobre la distribución marginal de los demás atributos. Para dos atributos puede visualizar interacciones, aunque la superficie es más difícil de estimar e interpretar.

La curva ICE (*individual conditional expectation*) conserva una línea por observación: $ICE_i(z) = \hat{f}(z, x_{i,-j})$. El PDP es su promedio. ICE revela heterogeneidad que el promedio oculta: un efecto creciente en una planta y decreciente en otra puede producir una línea plana. Centrar las curvas facilita comparar formas.

Si X_j está correlacionado con X_{-j} , sustituirlo globalmente crea casos improbables. Por ejemplo, combinar una elevada producción con configuraciones de maquinaria usadas solo a baja producción. Debe marcarse la densidad de datos, restringir el rango, usar perfiles acumulados locales o análisis condicional, y no interpretar regiones sin soporte.

Para clasificación, conviene indicar si se grafica puntuación, logit o probabilidad; el promedio en cada escala difiere. Las curvas no son causales: mantienen matemáticamente fijos atributos que en el mundo podrían cambiar juntos. Tampoco cuantifican importancia; una curva con gran amplitud puede corresponder a una región rara.

Un buen informe incluye distribución del atributo, intervalos por remuestreo, ICE o subgrupos, y comparación entre periodos. Saltos abruptos pueden reflejar árboles, pocos datos o una interacción. Se investigan antes de traducirlos en política.

12.4.4. Valores SHAP

Los valores SHAP se inspiran en valores de Shapley de teoría de juegos. Los atributos son “jugadores”, la predicción es el pago y cada contribución promedia el aporte marginal del atributo sobre todas las coaliciones posibles. Para p atributos,

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [\nu(S \cup \{j\}) - \nu(S)],$$

donde $\nu(S)$ es la predicción esperada cuando se conocen atributos de S . La descomposición satisface aditividad local:

$$\hat{f}(x) = \phi_0 + \sum_{j=1}^p \phi_j,$$

con ϕ_0 como valor esperado de referencia, en la escala explicada.

El cálculo exacto requiere 2^P coaliciones en general; se usan algoritmos especializados o aproximaciones. La definición de “atributo ausente” es decisiva. Un enfoque marginal integra sobre la distribución de los otros atributos y puede generar combinaciones irreales; uno condicional respeta dependencias, pero reparte crédito entre correlacionados de manera dependiente del modelo condicional. No existe una única atribución neutral.

Ejemplo: un modelo de riesgo de reclamación produce log-odds base -2.0 . Para una transacción, importe atípico aporta $+0.8$, velocidad $+0.5$, antigüedad de cuenta -0.3 y otros $+0.1$. La suma -0.9 se transforma a probabilidad logística aproximada 0.289 . Las contribuciones explican el desplazamiento del modelo desde la referencia; no son cambios causales de probabilidad ni necesariamente están expresadas en puntos porcentuales.

Agregando $|\phi_j|$ se obtiene una relevancia global basada en explicaciones locales. Los gráficos de dependencia SHAP ayudan a observar forma e interacción. Los riesgos incluyen seleccionar una referencia inadecuada, ocultar correlación, confundir escala y construir una historia plausible para un caso fuera de distribución. Siempre deben declararse datos de fondo, versión del modelo y estabilidad ante referencias alternativas.

12.4.5. Motivación del aprendizaje federado

En aprendizaje centralizado, los datos de distintas fuentes se trasladan a un repositorio común. Esto puede ser inviable por regulación, confidencialidad, soberanía, volumen o latencia. El aprendizaje federado (FL) coordina entrenamiento entre clientes que conservan datos locales y comparten actualizaciones del modelo. Ejemplos incluyen plantas de empresas distintas, instituciones financieras o dispositivos energéticos.

El objetivo idealizado con K clientes es

$$\min_w F(w) = \sum_{k=1}^K \frac{n_k}{N} F_k(w), \quad F_k(w) = \frac{1}{n_k} \sum_{i \in D_k} \ell(w; x_i, y_i),$$

donde $N = \sum_k n_k$. En *Federated Averaging* (FedAvg), el servidor distribuye w_t , clientes seleccionados realizan varias actualizaciones locales y devuelven $w_{t+1}^{(k)}$. El servidor agrega

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{r \in S_t} n_r} w_{t+1}^{(k)}.$$

Pseudocódigo: FedAvg

```

inicializar modelo global w
para cada ronda t:
    seleccionar clientes disponibles S_t
    enviar w a cada cliente seleccionado
    en cada cliente, en paralelo:
        entrenar localmente durante E pasos sobre sus datos

```

devolver actualización protegida
verificar y agregar actualizaciones ponderadas
evaluar el modelo global con protocolo federado

FL minimiza transferencia de datos brutos, pero no garantiza privacidad. Gradientes y parámetros pueden filtrar información. Tampoco resuelve automáticamente sesgo, consentimiento ni mala calidad. Su adopción se justifica tras comparar con alternativas como entrenamiento local, datos agregados, enclaves seguros o centralización autorizada.

12.4.6. Arquitecturas federadas

La federación horizontal ocurre cuando clientes comparten el espacio de atributos pero poseen filas distintas: varias plantas registran las mismas señales para máquinas diferentes. En federación vertical, organizaciones comparten entidades pero tienen atributos distintos, como una entidad financiera y una aseguradora con bases legales para colaborar. La transferencia federada aborda escasa coincidencia tanto de filas como de atributos y es más compleja.

Según escala se distingue *cross-silo*, con pocas organizaciones relativamente estables, de *cross-device*, con muchos dispositivos intermitentes y limitados. Cross-silo facilita control de versiones y conectividad; cross-device exige selección masiva, tolerancia a fallos y compresión. Una topología central usa servidor coordinador; una descentralizada intercambia actualizaciones entre pares, reduciendo un punto único pero complicando consenso y confianza.

La personalización reconoce que un único modelo global puede ser inadecuado. Puede ajustarse una cabeza local, agrupar clientes similares, usar regularización hacia un centro global o aprender representaciones compartidas. La evaluación debe comparar modelo global, local y personalizado por cliente; un promedio favorable puede ocultar daño en participantes pequeños.

Una arquitectura incluye más que agregación: identidad y autorización, registro de versiones, selección de clientes, distribución segura, entrenamiento local reproducible, agregación, monitoreo y recuperación. El esquema de atributos y la semántica deben alinearse. “Temperatura máxima” medida en ventanas distintas no se vuelve comparable por federar.

La partición de datos determina el protocolo. En vertical se requieren técnicas de alineación privada de entidades y cálculo seguro. En horizontal, el desafío dominante suele ser distribución no idéntica. La arquitectura se elige por estructura, amenaza y gobernanza, no por popularidad algorítmica.

12.4.7. Privacidad, comunicación y limitaciones

Las actualizaciones pueden revelar pertenencia, ejemplos o propiedades del cliente. La agregación segura permite que el servidor vea solo la suma, no contribuciones individuales. La privacidad diferencial limita influencia individual mediante recorte y ruido, y proporciona un presupuesto (ϵ, δ) ; mayor privacidad suele reducir utilidad. El cifrado homomórfico y el cómputo multipartito ofrecen protecciones adicionales con costo de cálculo y comunicación.

El modelo de amenaza debe especificar adversarios: servidor curioso, clientes maliciosos, observadores de red o colusión. Agregación segura no impide que un

cliente envenene el modelo; privacidad diferencial no garantiza equidad; cifrado en tránsito no protege actualizaciones una vez descifradas. Se requieren autenticación, agregación robusta, límites de actualización, auditoría y respuesta a incidentes.

La heterogeneidad estadística (*non-IID*) aparece cuando prevalencias o relaciones difieren entre clientes. Múltiples pasos locales pueden hacer que modelos deriven en direcciones incompatibles. Clientes grandes dominan un promedio por n_k , mientras ponderación uniforme puede sobrerrepresentar sitios pequeños. La función objetivo debe reflejar equidad y propósito, no solo tamaño.

La comunicación puede dominar el costo. Número de rondas, tamaño del modelo, disponibilidad, ancho de banda y energía condicionan viabilidad. Compresión, cuantización, actualizaciones dispersas y más trabajo local reducen transferencia, pero pueden afectar convergencia. Clientes lentos generan *stragglers*; excluirlos sistemáticamente sesga la población participante.

Otros límites son deriva asincrónica, borrado de datos difícil de propagar, reproducibilidad, depuración sin acceso central y cumplimiento entre jurisdicciones. La evaluación necesita métricas globales y por cliente, tasa de participación, rondas, bytes, consumo, ataques simulados y presupuesto de privacidad. Decir que “los datos nunca salen” es insuficiente y potencialmente engañoso: sale información transformada cuyo riesgo debe medirse.

12.4.8. Ejemplo práctico guiado: explicación de una predicción individual

Un ensamble de Gradient Boosting predice la probabilidad de que una pieza de una línea de plantas requiera reproceso. El modelo se ha validado por planta y periodo. Para una pieza concreta devuelve 0.68, frente a una prevalencia de referencia de 0.12. La decisión es inspeccionar cuando la probabilidad supera 0.45.

Explicación local. En escala log-odds, SHAP usa una referencia de -1.99 (probabilidad 0.12). Vibración aporta $+1.10$, variación térmica $+0.65$, proveedor del material $+0.35$, velocidad de línea $+0.22$ y humedad ambiental -0.18 ; otros aportes suman $+0.61$. El total 0.76 corresponde aproximadamente a 0.68. Se comunica que son contribuciones del modelo respecto de la población de fondo, no causas independientes.

Contraste global. La permutación muestra que vibración es primera, velocidad segunda, variación térmica cuarta y proveedor séptimo. No hay contradicción: proveedor puede ser poco importante en promedio y relevante para este caso. Se repite permutación por planta; proveedor gana importancia en la planta de la pieza, lo que sugiere heterogeneidad a investigar.

Forma y soporte. Un PDP de vibración crece a partir de cierto rango, pero las curvas ICE revelan que el aumento es mayor a velocidades altas. La pieza se encuentra dentro de una región con suficientes observaciones. Se evita afirmar que reducir vibración por sí sola llevará la probabilidad a un valor específico, pues alterar una variable puede crear una combinación no realista y no establece intervención causal.

Sensibilidad. Se recalculan SHAP con dos fondos razonables: toda la red y la planta local. Cambian magnitudes, no el conjunto principal. Pequeñas perturbaciones dentro del error de sensor mantienen la decisión. Si la explicación cambiara de signo por ruido mínimo, se marcaría baja estabilidad.

Vínculo federado. Si las plantas entrenan federadamente, el modelo global puede explicarse localmente en cada sitio sin centralizar filas. Sin embargo, compartir explicaciones detalladas también puede revelar propiedades. Se agregan estadísticas con umbrales de tamaño y controles de privacidad. El informe final contiene puntuación, umbral, referencia, contribuciones, soporte, advertencias y versión.

Este ejercicio muestra que una explicación responsable triangula escala local, importancia global, forma promedio, subgrupos y estabilidad. Ninguna herramienta aislada responde todas las preguntas.

Síntesis

Los árboles convierten el espacio de atributos en regiones mediante divisiones voraces. Entropía y Gini cuantifican impureza; la pérdida cuadrática guía regresión. Profundidad, tamaño de hoja y poda costo-complejidad controlan el compromiso sesgo-varianza. Sus reglas son legibles, pero siguen siendo asociaciones sensibles a muestra, medición y cobertura.

Las SVM eligen hiperplanos de margen máximo. El margen blando expresa la regularización mediante pérdida bisagra y C ; los kernels reemplazan productos internos para obtener fronteras no lineales. Escalado, búsqueda conjunta, calibración y costo son partes del sistema. Los vectores de soporte revelan qué casos condicionan la frontera, no qué casos deben eliminarse.

Los ensambles explotan diversidad. Bagging y Random Forest reducen varianza; boosting optimiza secuencialmente una pérdida; stacking combina modelos mediante predicciones fuera de pliegue. Una comparación válida comparte datos, divisiones, métricas y presupuesto, e incluye estabilidad y costo. El ganador de entrenamiento no es el objetivo: lo es el sistema que generaliza bajo restricciones reales.

Permutación, PDP y SHAP responden preguntas diferentes y dependen de correlación, referencia y población. Explicar un modelo no identifica causas. El aprendizaje federado descentraliza datos brutos, pero agrega heterogeneidad, comunicación y amenazas. Privacidad requiere mecanismos y un modelo explícito, no solo una arquitectura distribuida.

Glosario

- **Agregación segura:** protocolo que permite combinar actualizaciones sin revelar al coordinador cada contribución individual.
- **Bagging:** ajuste de modelos sobre remuestras y agregación de sus predicciones para reducir varianza.
- **Boosting:** construcción secuencial de modelos que corrigen errores o gradientes previos.
- **Calibración:** correspondencia entre probabilidades predichas y frecuencias observadas.
- **Dependencia parcial:** efecto promedio del modelo al fijar uno o más atributos y promediar sobre los restantes.
- **Entropía:** medida de incertidumbre de una distribución de clases.
- **Ganancia de información:** reducción de entropía producida por una división.
- **Gradient Boosting:** boosting entendido como descenso de gradiente en el espacio de funciones.

- **Índice de Gini:** medida de impureza basada en probabilidades cuadráticas de clase.
- **Kernel:** función que calcula un producto interno implícito en un espacio de características.
- **Margen:** separación geométrica entre una frontera SVM y los casos más próximos.
- **Non-IID:** distribuciones locales que no son idénticas ni necesariamente independientes entre clientes.
- **Out-of-bag:** observación no incluida en una remuestra bootstrap concreta.
- **Out-of-fold:** predicción generada por un modelo que no entrenó con esa observación.
- **Poda:** eliminación regularizada de ramas de un árbol.
- **Privacidad diferencial:** garantía probabilística que limita cuánto cambia una salida por la presencia de una observación.
- **Random Forest:** bagging de árboles con selección aleatoria de atributos en cada nodo.
- **SHAP:** atribuciones basadas en valores de Shapley que descomponen una predicción respecto de una referencia.
- **Stacking:** combinación aprendida de predicciones OOF mediante un meta-modelo.
- **Vector de soporte:** observación con multiplicador dual positivo que participa en la frontera SVM.

Preguntas de revisión y discusión

1. ¿Por qué una partición que produce una hoja pura puede tener poca ganancia de información?
2. Derive la constante óptima de una hoja de regresión bajo pérdida cuadrática y explique qué cambiaría con pérdida absoluta.
3. Compare Gini y entropía. ¿Qué factores suelen ser más importantes que elegir entre ambos?
4. ¿Por qué un árbol profundo puede tener bajo error de entrenamiento y mala calibración futura?
5. Explique la relación entre poda costo-complejidad, α y número de hojas.
6. ¿Qué información debe acompañar una regla de árbol para evitar una interpretación engañosa?
7. Derive la distancia de un punto a un hiperplano y la anchura $2/\|w\|$ del margen.
8. Interprete $\xi_i = 0.4$ y $\xi_i = 1.4$ en una SVM de margen blando.
9. ¿Cómo interactúan C y γ en una SVM RBF?
10. ¿Por qué el escalado debe estimarse dentro de cada pliegue?
11. Use la fórmula de varianza del promedio para explicar el propósito de m_{try} en Random Forest.
12. ¿Cuándo la estimación OOB puede ser optimista o inadecuada?
13. Relacione los seudoresiduos de Gradient Boosting con descenso de gradiente.
14. Dibuje un caso donde boosting persiga etiquetas erróneas.
15. ¿Por qué las predicciones de nivel 0 de stacking deben ser OOF?
16. Distinga importancia por disminución de impureza e importancia por permutación.
17. ¿Cómo afectan predictores correlacionados a permutación, PDP y SHAP?
18. ¿Por qué una explicación local fiel no constituye una explicación causal?

19. Compare federación horizontal, vertical, cross-silo y cross-device.
20. ¿Qué amenazas permanecen si se utiliza agregación segura?
21. Proponga métricas para evaluar utilidad, equidad, privacidad y comunicación en FL.
22. ¿Qué señales indicarían deriva y qué componentes deberían reajustarse?

Actividad integradora

Título: priorización de inspecciones de calidad en una red de plantas.

Una organización dispone de registros históricos de varias plantas: mediciones previas al cierre de lote, tipo de producto, proveedor, turno, antigüedad de máquina y etiqueta de defecto crítico confirmada. Cada planta conserva sus datos por acuerdos de confidencialidad. La capacidad permite inspeccionar como máximo el 8% de los lotes.

El equipo debe entregar un diseño, sin código de implementación, que contenga:

1. Una formulación predictiva con unidad de análisis, tiempo de decisión, horizonte, acción, costo de falsos negativos y variables prohibidas por temporalidad.
2. Una división de desarrollo, validación y prueba que respete periodo, planta y posibles lotes relacionados.
3. La derivación manual de Gini o entropía para dos cortes candidatos en un nodo proporcionado por el docente.
4. Un árbol amplio y su estrategia conceptual de poda, incluyendo la regla de selección por validación.
5. Una SVM lineal y una RBF, con justificación de escalado, malla logarítmica, calibración y diagnóstico de soportes.
6. Una competencia entre árbol podado, Random Forest y Gradient Boosting bajo idéntico protocolo, con tabla de media, dispersión, sensibilidad al 8%, calibración, latencia y memoria.
7. Un diseño opcional de stacking que detalle exactamente cómo producir predicciones OOF y cómo evitar fuga.
8. Importancia por permutación global y por planta, un PDP acompañado de ICE y una explicación SHAP individual con dos poblaciones de referencia.
9. Una crítica explícita de correlación, combinaciones sin soporte, inestabilidad y límites causales.
10. Una arquitectura federada cross-silo con esquema de FedAvg, gestión de clientes, agregación segura, privacidad diferencial si corresponde, amenazas y presupuesto de comunicación.
11. Un plan de monitoreo con métricas por planta, deriva, calibración, participación, incidentes de seguridad y condiciones de retirada del modelo.
12. Una recomendación final que identifique un modelo elegido y otro descartado, cuantifique el intercambio y declare incertidumbre y limitaciones.

La evaluación de la actividad ponderará corrección metodológica, ausencia de fuga, coherencia entre métrica y decisión, calidad de diagnósticos, interpretación prudente y viabilidad distribuida. No se premiará el modelo más complejo, sino la argumentación reproducible que conecte datos, objetivo, validación y operación.